

PRODUCT DEFINITION

# Numo

Personal Budget iOS App

Product Brief, UX Specification, User Flows, and Technical Direction

|                  |                                                      |
|------------------|------------------------------------------------------|
| App name         | Numo                                                 |
| Version          | 1.0                                                  |
| Status           | Product Definition                                   |
| Platform         | iOS                                                  |
| Primary market   | Tunisia / North Africa initially                     |
| Primary currency | TND                                                  |
| Primary language | English for V1                                       |
| Design direction | Native iOS, flat, minimal, fully rounded             |
| Architecture     | SwiftUI + SwiftData + Supabase                       |
| Backend          | Hono, introduced where server-side logic is required |

*Record spending in seconds. Understand your money at a glance.*

# Contents

|                                         |    |                                           |    |
|-----------------------------------------|----|-------------------------------------------|----|
| 1. Product Overview . . . . .           | 2  | 42. Recurring Expenses . . . . .          | 28 |
| 2. Product Philosophy . . . . .         | 2  | 43. Notifications . . . . .               | 29 |
| 3. Target User . . . . .                | 4  | 44. Settings . . . . .                    | 29 |
| 4. Core Product Concept . . . . .       | 4  | 45. Export . . . . .                      | 30 |
| 5. Primary Navigation . . . . .         | 4  | 46. Offline Scenario . . . . .            | 31 |
| 6. Design System . . . . .              | 5  | 47. Sync Conflict Scenario . . . . .      | 31 |
| 7. Color System . . . . .               | 6  | 48. Authentication . . . . .              | 32 |
| 8. Typography . . . . .                 | 7  | 49. Privacy . . . . .                     | 32 |
| 9. Corner Radius . . . . .              | 7  | 50. Search . . . . .                      | 32 |
| 10. Core Data Model . . . . .           | 8  | 51. Apple Ecosystem Flow . . . . .        | 33 |
| 11. Local Data Layer . . . . .          | 10 | 52. NFC Scenarios . . . . .               | 34 |
| 12. Backend . . . . .                   | 10 | 53. Apple Watch, Future . . . . .         | 34 |
| 13. First Launch Flow . . . . .         | 11 | 54. Dynamic Island Product Rule . . . . . | 35 |
| 14. Onboarding Rules . . . . .          | 12 | 55. Empty States . . . . .                | 35 |
| 15. Home Screen . . . . .               | 12 | 56. Error States . . . . .                | 36 |
| 16. Add Expense Flow . . . . .          | 13 | 57. Loading States . . . . .              | 36 |
| 17. Amount Input . . . . .              | 14 | 58. Performance Requirements . . . . .    | 36 |
| 18. Category Selection . . . . .        | 14 | 59. Accessibility . . . . .               | 37 |
| 19. Optional Note . . . . .             | 15 | 60. Haptics . . . . .                     | 37 |
| 20. Save Expense . . . . .              | 15 | 61. Animations . . . . .                  | 38 |
| 21. Voice Expense Flow . . . . .        | 16 | 62. Core User Journey . . . . .           | 39 |
| 22. Siri . . . . .                      | 17 | 63. Power User Journey . . . . .          | 39 |
| 23. App Intents . . . . .               | 17 | 64. Daily Usage Scenario . . . . .        | 40 |
| 24. Shortcuts . . . . .                 | 18 | 65. Monthly Review Scenario . . . . .     | 41 |
| 25. Control Center . . . . .            | 19 | 66. Mistake Scenario . . . . .            | 41 |
| 26. Action Button . . . . .             | 19 | 67. Wrong Category Scenario . . . . .     | 42 |
| 27. Dynamic Island . . . . .            | 19 | 68. No Internet Scenario . . . . .        | 42 |
| 28. Lock Screen Widget . . . . .        | 20 | 69. New Month Scenario . . . . .          | 42 |
| 29. Home Screen Widgets . . . . .       | 20 | 70. Product Boundaries . . . . .          | 43 |
| 30. Activity Screen . . . . .           | 21 | 71. V1 Feature Set . . . . .              | 44 |
| 31. Expense Detail Flow . . . . .       | 22 | 72. Technical Architecture . . . . .      | 45 |
| 32. Edit Expense Flow . . . . .         | 23 | 73. Local-First Sync Model . . . . .      | 46 |
| 33. Delete Expense Flow . . . . .       | 23 | 74. API Principles . . . . .              | 46 |
| 34. Income Flow . . . . .               | 24 | 75. Security . . . . .                    | 47 |
| 35. Budget Calculation . . . . .        | 24 | 76. Monetization Direction . . . . .      | 47 |
| 36. Budget Behavior . . . . .           | 25 | 77. Analytics . . . . .                   | 47 |
| 37. Monthly Reset . . . . .             | 25 | 78. Success Metrics . . . . .             | 48 |
| 38. Insights Screen . . . . .           | 26 | 79. UX Quality Bar . . . . .              | 49 |
| 39. Spending Patterns . . . . .         | 26 | 80. Product Identity . . . . .            | 49 |
| 40. Natural Language Insights . . . . . | 27 | 81. Final UX Rule . . . . .               | 50 |
| 41. Automatic Categories . . . . .      | 27 |                                           |    |

# 1. Product Overview

The product is an extremely simple personal budget and expense tracking application designed around one principle:

*Record spending in seconds. Understand your money at a glance.*

The application should remove the friction found in traditional budgeting applications.

The user should not need to:

- Create multiple financial accounts
- Configure complex budgets
- Create categories before using the application
- Enter payment methods
- Enter bank information
- Fill long forms
- Understand financial terminology
- Navigate through multiple screens to record an expense

The primary action is:

```
Open
→ Enter amount
→ Save
```

The application should feel like a native iOS utility rather than a traditional finance dashboard.

## 2. Product Philosophy

### Core Principles

#### 2.1 Speed

Recording an expense should take approximately 3 seconds.

Target interaction:

```
Tap
→ Enter amount
→ Save
```

Voice input should reduce the interaction further:

```
"Spent 12 dinars on lunch"
→ 12 DT
→ Food
→ Lunch
→ Saved
```

## 2.2 Simplicity

Every feature must justify its presence.

If a feature requires significant configuration, it should not be part of the core experience.

## 2.3 Local-first

The application should work without an internet connection.

Expenses should save locally immediately.

Synchronization should happen in the background.

## 2.4 iOS-first

The application should use native iOS capabilities as product features rather than treating them as optional integrations.

Target integrations:

- Siri
- Shortcuts
- App Intents
- Spotlight
- Widgets
- Lock Screen
- Control Center
- Action Button
- Dynamic Island
- Live Activities
- Share Sheet
- Apple Watch in a later version

## 2.5 Minimal visual complexity

The application should prioritize information hierarchy over decoration.

Avoid:

- Complex dashboards
- Excessive charts
- Gradients
- Neumorphism
- Glassmorphism
- 3D elements
- Decorative illustrations
- Excessive colors
- Excessive shadows
- Dense financial tables

---

## 3. Target User

The primary user is someone who wants to understand and control personal spending without maintaining a complex financial system.

Typical user behavior:

- Receives monthly income
- Has regular monthly expenses
- Makes frequent small purchases
- Wants to know where money goes
- Wants to avoid overspending
- Does not want to manually maintain spreadsheets
- Uses an iPhone daily
- Values fast interactions
- Wants the application to work without complicated configuration

The product is not initially designed for:

- Professional accounting
- Business bookkeeping
- Investment management
- Tax management
- Multi-company accounting
- Advanced financial planning

---

## 4. Core Product Concept

The application revolves around four concepts:

Money available  
Money spent  
Money received  
Money remaining

The user should understand these without learning a financial system.

---

## 5. Primary Navigation

The application has four primary destinations:

Home  
Activity  
Insights  
Settings

Navigation uses a floating bottom navigation bar.

The navigation bar:

- Does not touch the screen edges
- Has fully rounded corners
- Floats above the bottom safe area
- Uses approximately 16–20 px horizontal margins
- Uses approximately 12–16 px bottom spacing
- Uses subtle contrast
- Uses SF Symbols-style icons
- Uses the accent color for the selected destination

Primary expense action:

+ Expense

The expense action must remain visually stronger than the navigation.

---

## 6. Design System

### 6.1 Design Style

The visual direction is:

*Native iOS minimalism with flat surfaces, fully rounded corners, strong typography, and selective translucency only where system UI benefits from it.*

Primary characteristics:

- Flat
- Rounded
- Minimal
- Spacious
- Native
- Calm
- Functional

---

## 7. Color System

### Light Mode

Background:  
#F5F5F7

Primary Surface:  
#FFFFFF

Primary Text:  
#111111

Secondary Text:  
#707070

Muted Text:  
#A0A0A5

Divider:  
#E8E8EA

Primary Accent:  
#4F6BFF

Success:  
Muted green

Destructive:  
Muted red

### Dark Mode

Background:  
#000000

Primary Surface:  
#1C1C1E

Secondary Surface:  
#2C2C2E

Primary Text:  
#FFFFFF

Secondary Text:  
#98989D

Accent:  
#6F82FF

The accent color should be used selectively.

Do not color every category differently.

Financial amounts should generally remain neutral.

Green should represent actual positive changes, not every monetary value.

---

## 8. Typography

Use Apple's SF Pro typography system.

Hierarchy:

Large Financial Value  
32–44 pt

Primary Heading  
28–34 pt

Section Heading  
20–24 pt

Primary Body  
16–17 pt

Secondary Body  
14–15 pt

Caption  
12–13 pt

Financial numbers should receive strong visual hierarchy.

Example:

1,240 DT  
left this month

The amount should dominate the screen.

---

## 9. Corner Radius

All major interface elements use rounded corners.

Recommended:

Large cards:  
24–28 px

Buttons:  
18–24 px

Input fields:  
18–22 px

Small controls:  
14–18 px

Floating navigation:  
24–28 px

Avoid sharp rectangular UI.

---

## 10. Core Data Model

### User

```
User
- id
- email
- createdAt
- currency
- timezone
```

### Budget

```
Budget
- id
- userId
- amount
- currency
- resetDay
- createdAt
- updatedAt
```

## Expense

```
Expense
- id
- userId
- amount
- categoryId
- note
- spentAt
- createdAt
- updatedAt
```

## Income

```
Income
- id
- userId
- amount
- source
- receivedAt
- createdAt
```

## Category

```
Category
- id
- userId
- name
- icon
- isDefault
- createdAt
```

Default categories:

```
Food
Transport
Shopping
Bills
Entertainment
Health
Other
```

The user should not need to configure these.

---

## 11. Local Data Layer

Use SwiftData for local persistence.

Primary requirement:

```
Create expense
→ Save locally
→ Update UI immediately
→ Sync later
```

Never make the primary expense flow dependent on network availability.

---

## 12. Backend

Recommended architecture:

```
SwiftUI
|
+-- SwiftData
|
+-- App Intents
|
+-- WidgetKit
|
+-- ActivityKit
|
v
Hono API
|
v
Supabase
|
+-- PostgreSQL
+-- Auth
+-- Storage
```

Hono should remain small.

Do not create backend endpoints for logic that belongs naturally inside the iOS application.

Use Hono for:

- Authentication-related server operations where needed
- Server-side business logic
- AI processing
- Webhooks
- External integrations
- Secure operations
- Subscription validation
- Future server-side analytics

Supabase handles:

- PostgreSQL
- Authentication
- Row Level Security
- Storage
- Data synchronization infrastructure

---

## 13. First Launch Flow

### Scenario

The user installs the application for the first time.

#### Step 1

Display:

Your money.  
Made simple.

Secondary text:

Track your spending without the spreadsheets.

Primary action:

Get started

#### Step 2

Ask:

How much do you want to manage each month?

Amount input:

1,500 DT

#### Step 3

Ask:

When should your budget reset?

Options:

1st of every month  
15th of every month  
Custom

Default:

1st of every month

#### Step 4

Finish:

You're ready.

Primary action:

Start budgeting

The user enters Home.

---

## 14. Onboarding Rules

Do not ask during onboarding for:

- Bank accounts
- Credit cards
- Payment methods
- Financial institutions
- Categories
- Spending limits by category
- Income sources
- Investment information
- Receipts
- Recurring transactions
- Notifications

The user should reach the main application as quickly as possible.

---

## 15. Home Screen

### Purpose

Answer:

1. How much money do I have left?
2. How much have I spent today?
3. How much should I spend today?

## Layout

Good morning

1,240 DT  
left this month

32 DT  
spent today

62 DT  
daily allowance

Today

Food

12 DT

Transport

5 DT

Coffee

3 DT

This month

Food

420 DT

Transport

210 DT

Shopping

180 DT

Other

430 DT

Floating bottom navigation:

Home

Activity

Insights

Settings

Primary action:

+ Expense

## 16. Add Expense Flow

### Standard Flow

Home

↓

+ Expense

↓

Amount input

↓

Category

↓

Save

↓

Home

Example:

```
+ Expense
↓
12
↓
Food
↓
Save
```

Total interaction should remain extremely short.

---

## 17. Amount Input

The amount should dominate the screen.

Example:

```
12.500 DT
```

Numeric keypad:

```
1  2  3
4  5  6
7  8  9
.  0  <=>
```

The user should never need to select the currency manually during normal usage.

Currency comes from the user's settings.

---

## 18. Category Selection

Categories appear as rounded controls.

Example:

```
Food
Transport
Shopping
Other
```

The most frequently used categories should appear first.

The application should learn from previous transactions.

Example:

User frequently records:  
Lunch  
→ Food

Future entries should default to Food when context supports it.

The user must always be able to change the category.

---

## 19. Optional Note

The note field is optional.

Placeholder:

Add a note

Never force the user to enter a description.

---

## 20. Save Expense

After tapping:

Save expense

The app should:

1. Save locally
2. Update the balance
3. Update today's spending
4. Update daily allowance
5. Update widgets
6. Update Live Activity if active
7. Queue synchronization
8. Return the user to the previous context

The user should see immediate confirmation.

Example:

12 DT added

Do not show a blocking loading screen.

---

## 21. Voice Expense Flow

The application should support natural language.

Example:

"Spent 12 dinars on lunch"

Expected result:

Amount:  
12 DT

Category:  
Food

Note:  
Lunch

Another example:

"Paid 8 dinars for a taxi"

Result:

8 DT  
Transport  
Taxi

If the parser is uncertain:

12 DT

What was this for?

Food  
Transport  
Shopping  
Other

The application should ask only for missing information.

---

## 22. Siri

Supported commands:

Add an expense

Add 12 dinars to my expenses

I spent 12 dinars on food

How much did I spend today?

How much money do I have left?

How much can I spend today?

---

## 23. App Intents

Expose the following intents:

Add Expense  
Add Income  
Get Today's Spending  
Get Remaining Budget  
Get Daily Allowance  
Open Activity  
Open Insights

These intents should power:

- Siri
- Shortcuts
- Spotlight
- Widgets
- Control Center
- Action Button

---

## 24. Shortcuts

Provide predefined Shortcuts actions.

### Add Expense

Add Expense

Parameters:

Amount  
Category  
Note

### Check Spending

Check Today's Spending

Returns:

You spent 32 DT today.

### Check Remaining Budget

Check Remaining Budget

Returns:

You have 1,240 DT left this month.

### Check Daily Allowance

Check Daily Allowance

Returns:

You have 62 DT available for today.

---

## 25. Control Center

Provide a control:

+ Expense

Tap:

→ Open instant expense entry

The interaction should avoid opening the full application where iOS supports direct execution.

---

## 26. Action Button

On supported iPhone models, allow the user to assign:

Add Expense

Preferred behavior:

Action Button  
↓  
Amount input  
↓  
Save

The user should not need to browse the application.

---

## 27. Dynamic Island

Dynamic Island should remain optional.

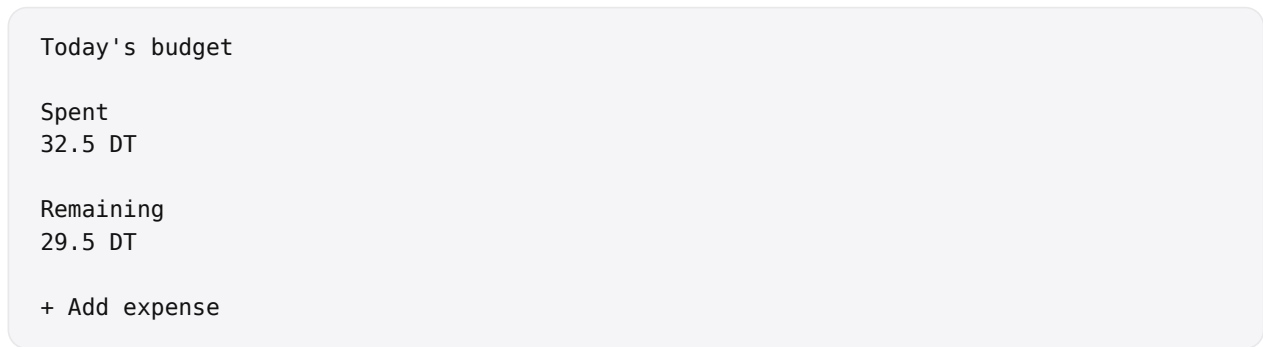
Do not permanently occupy the Dynamic Island.

Use Live Activities for temporary budget awareness.

Example:

29.5 DT  
left today

Expanded view:



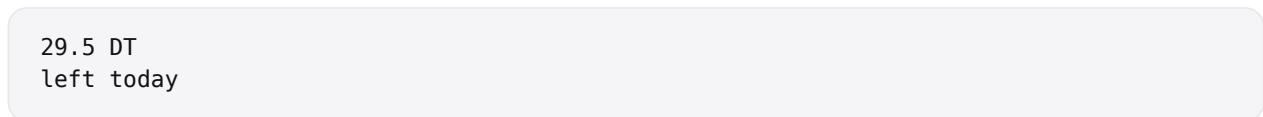
The Live Activity should disappear or become inactive when no longer useful.

Do not force Dynamic Island usage simply because the device supports it.

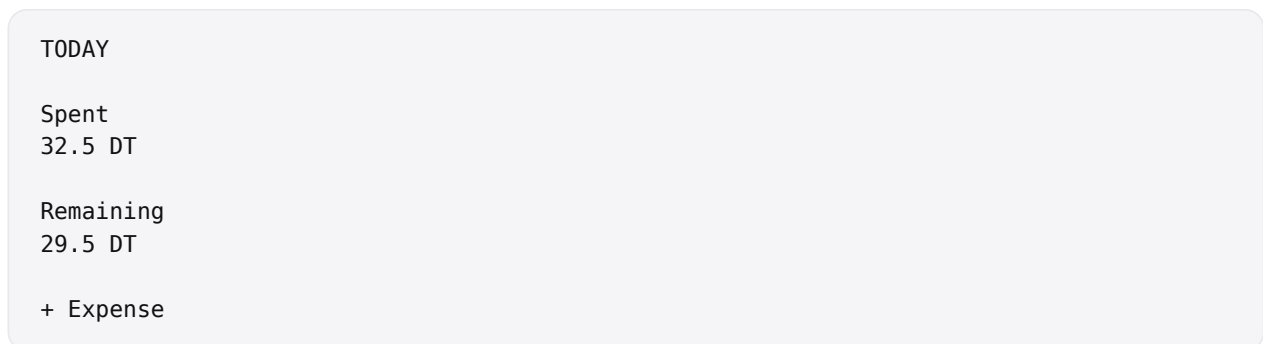
---

## 28. Lock Screen Widget

Small widget:



Medium widget:



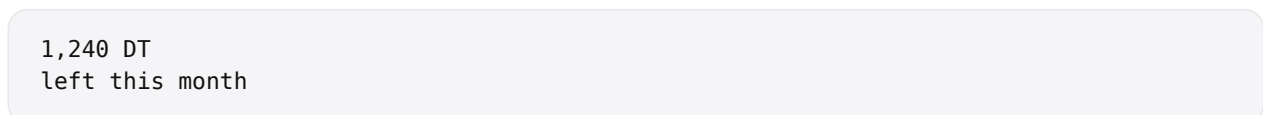
The widget should provide direct access to expense entry.

---

## 29. Home Screen Widgets

Provide a small number of widget sizes.

### Small



## Medium

SEPTEMBER

1,240 DT left

Today  
32 DT spent

## Interactive

+ Expense

Do not create many widget variants.

# 30. Activity Screen

## Purpose

Show spending history.

Top:

Activity

September 2026

Summary:

1,240 DT  
total spent

Transactions grouped by date.

Example:

Today

|        |       |
|--------|-------|
| Lunch  | 12 DT |
| Taxi   | 5 DT  |
| Coffee | 3 DT  |

Yesterday

|           |       |
|-----------|-------|
| Groceries | 34 DT |
| Coffee    | 4 DT  |

Transaction rows contain:

Icon

Category

Note

Amount

Tap a transaction to open details.

## 31. Expense Detail Flow

User taps:

Lunch

12 DT

Open a rounded sheet:

12 DT

Food

Lunch

September 24

12:32

Edit

Delete

Edit should allow:

Amount

Category

Note

Date

Delete should require confirmation.

---

## 32. Edit Expense Flow

```
Expense detail
↓
Edit
↓
Change amount/category/note/date
↓
Save
↓
Update local data
↓
Sync
```

Changes should immediately update:

- Home
- Activity
- Insights
- Widgets
- Daily allowance
- Monthly balance

---

## 33. Delete Expense Flow

```
Expense detail
↓
Delete
↓
Confirm
```

Confirmation:

```
Delete this expense?

12 DT
Lunch

Cancel
Delete
```

After deletion:

```
Expense deleted
```

Balance and analytics update immediately.

---

## 34. Income Flow

Income should remain simpler than expense tracking.

Entry:

+ Income

Input:

1,500 DT

Optional source:

Salary  
Freelance  
Other

Save.

The application updates available money.

Income should not dominate the primary interface.

---

## 35. Budget Calculation

Basic calculation:

Remaining monthly budget  
=  
Budget amount  
- spending

Daily allowance:

Remaining budget  
/  
Remaining budget days

Example:

Budget:  
1,500 DT

Spent:  
260 DT

Remaining:  
1,240 DT

Remaining days:  
20

Daily allowance:  
62 DT

The exact calculation should account for the selected budget reset date.

---

## 36. Budget Behavior

The user should see a simple interpretation:

62 DT  
daily allowance

After spending 20 DT:

42 DT  
left today

If the user spends above the daily allowance, do not punish the user with aggressive warnings.

Instead:

You spent 8 DT above today's allowance.

The application should adjust future daily allowance based on the remaining budget.

---

## 37. Monthly Reset

At the beginning of a new budget period:

New budget period

The application automatically starts the new period.

Historical transactions remain accessible.

The user should not need to manually archive anything.

---

## 38. Insights Screen

Purpose:

Where did my money go?

Main statistic:

1,240 DT  
spent this month

Comparison:

85 DT less than last month

Category breakdown:

Food  
420 DT

Transport  
210 DT

Shopping  
180 DT

Other  
430 DT

Use horizontal bars.

Avoid pie charts.

---

## 39. Spending Patterns

Show simple information.

Example:

Daily average  
41 DT

Daily allowance  
62 DT

Weekly visualization can show:



Do not turn Insights into an analytics dashboard.

## 40. Natural Language Insights

The application can produce simple factual summaries.

Examples:

You spent 85 DT less than last month.

Food represents your largest spending category this month.

Your daily average is 41 DT.

Avoid judgmental language.

Avoid:

You're wasting too much money.

Use neutral language:

Food spending is 120 DT higher than last month.

## 41. Automatic Categories

The application should reduce manual categorization.

Examples:

```
Lunch
→ Food

Taxi
→ Transport

Cinema
→ Entertainment

Groceries
→ Food
```

The system should learn from corrections.

If the user repeatedly changes:

```
Coffee
Other
→ Food
```

future Coffee transactions should prefer Food.

The user remains in control.

---

## 42. Recurring Expenses

Recurring expenses should not require manual configuration initially.

If the application detects repeated transactions:

```
Netflix
13.99 DT
September

Netflix
13.99 DT
August

Netflix
13.99 DT
July
```

Show:

```
You pay Netflix regularly.

Track this automatically?
```

Options:

Track automatically  
Not now

The application should learn recurring expenses from actual behavior.

---

## 43. Notifications

Notifications should be minimal.

Useful notification:

You have 42 DT left for today.

Optional.

Avoid:

You haven't logged an expense.

Avoid excessive reminders.

The product should not become annoying.

---

## 44. Settings

Sections:

### Budget

Monthly budget  
1,500 DT

Budget reset  
1st of every month

Currency  
TND

### Automation

Siri & Shortcuts  
Widgets  
Notifications

## Appearance

Appearance  
System

Options:

System  
Light  
Dark

## Data

Export data  
Backup & Sync

## About

Privacy  
About

---

# 45. Export

Support simple export.

Formats:

CSV  
JSON

CSV example:

date,amount,category,note  
2026-09-24,12,Food,Lunch  
2026-09-24,5,Transport,Taxi

Export should not be part of the primary user flow.

---

## 46. Offline Scenario

User has no internet.

They add:

```
12 DT  
Lunch
```

Expected behavior:

```
Save locally  
↓  
UI updates  
↓  
Expense appears immediately  
↓  
Sync status remains internal
```

When internet returns:

```
Local data  
↓  
Sync  
↓  
Supabase
```

The user should not need to do anything.

---

## 47. Sync Conflict Scenario

If the same transaction changes on multiple devices:

Use the latest valid update based on:

```
updatedAt
```

Do not create duplicate expenses.

The sync layer should use stable UUIDs.

---

## 48. Authentication

Authentication should remain optional during initial usage where possible.

Preferred experience:

```
Install
↓
Set budget
↓
Use application
```

Account creation should appear when needed for:

- Sync
- Backup
- Multi-device usage
- Restore

Do not block the first expense behind account creation.

---

## 49. Privacy

Financial data is sensitive.

The product should prioritize:

- Local-first storage
- Secure authentication
- Supabase Row Level Security
- Minimal data collection
- No selling user data
- No unnecessary third-party analytics

Do not collect information that the product does not need.

---

## 50. Search

Spotlight should expose useful application actions.

Examples:

```
Budget
```

→ Show remaining budget.

```
Expenses
```

→ Open Activity.

---

Add expense

→ Open expense entry.

---

## 51. Apple Ecosystem Flow

The application should support multiple entry points.

### App

Open app  
→ + Expense  
→ Amount  
→ Category  
→ Save

### Siri

"Add 12 dinars for lunch"  
→ Expense created

### Shortcut

Shortcut  
→ Add Expense  
→ Save

### Control Center

Control  
→ Add Expense

### Action Button

Action Button  
→ Add Expense

### Lock Screen

Widget  
→ Add Expense

## Home Screen

Widget  
→ Add Expense

## NFC

Optional future feature:

Tap NFC tag  
→ Open expense entry  
→ Category preselected

## 52. NFC Scenarios

Example NFC tag near a coffee machine:

Coffee

Tap:

NFC  
↓  
Expense entry  
↓  
Coffee category selected  
↓  
Enter amount  
↓  
Save

Another tag:

Transport

The category is preselected.

NFC should remain an optional advanced feature.

## 53. Apple Watch, Future

Do not build a full Apple Watch application initially.

Potential interactions:

Add expense

Check today's spending

Check daily allowance

The Apple Watch experience should remain action-focused.

---

## 54. Dynamic Island Product Rule

Dynamic Island should never exist purely for marketing.

Use Live Activities only when the information provides meaningful glanceable value.

Example:

29.5 DT  
left today

Do not keep a permanent Live Activity running without user benefit.

---

## 55. Empty States

### Home

New user:

No expenses yet.  
  
Your spending will appear here.  
  
+ Add expense

### Activity

Nothing here yet.

### Insights

Add a few expenses  
to see your spending patterns.

Do not use large illustrations.

---

## 56. Error States

Keep errors human-readable.

Bad:

```
HTTP 401  
SYNC_FAILED
```

Good:

```
We couldn't sync your latest changes.  
  
Your expenses are saved on this iPhone.  
We'll try again later.
```

---

## 57. Loading States

Avoid full-screen loading states.

Use skeletons only when necessary.

For local data:

```
Render immediately.
```

For synchronization:

```
Sync silently.
```

---

## 58. Performance Requirements

Primary actions should feel instant.

Target:

Expense save:  
Immediate local UI update

Home:  
Fast launch

Activity:  
Instant local rendering

Widget:  
Fast data refresh

Voice:  
Minimal processing delay

The application should prioritize perceived responsiveness.

---

## 59. Accessibility

Support:

- Dynamic Type
- VoiceOver
- Reduce Motion
- Increased Contrast
- Dark Mode
- Large text
- Haptic feedback

Do not rely on color alone to communicate meaning.

Buttons need readable labels.

Icons need accessibility labels.

---

## 60. Haptics

Use haptics sparingly.

Examples:

Expense saved:

Light success haptic

Delete:

Subtle warning haptic

Primary button:

Light selection haptic

Do not add haptics to every interaction.

---

## 61. Animations

Animations should be short and functional.

Examples:

Expense saved:

Amount  
↓  
small confirmation  
↓  
Home updates

Balance change:

1,252 DT  
↓  
1,240 DT

Use subtle number transitions.

Avoid:

- Confetti
- Bouncy cards
- Excessive spring animations
- Decorative transitions

---

## 62. Core User Journey

The ideal journey:

```
Install
↓
Set monthly budget
↓
Home
↓
Spend money
↓
Add expense
↓
Amount
↓
Category
↓
Save
↓
Continue using phone
```

The user should not need to think about the application after recording the expense.

---

## 63. Power User Journey

```
Action Button
↓
Add expense
↓
12 DT
↓
Food
↓
Done
```

Or:

```
Siri
↓
"Spent 12 dinars on lunch"
↓
Done
```

Or:

Control Center  
↓  
+ Expense  
↓  
12 DT  
↓  
Done

---

## 64. Daily Usage Scenario

Morning:

Open Home  
  
1,240 DT left  
62 DT daily allowance

Lunch:

"Spent 12 dinars on lunch"

Application:

12 DT  
Food  
Lunch

Afternoon:

Widget:  
  
50 DT  
left today

Evening:

Home:  
  
32 DT  
spent today  
  
30 DT  
left today

No manual calculations required.

# 65. Monthly Review Scenario

User opens Insights.

September

1,240 DT spent

Food

420 DT

Transport

210 DT

Shopping

180 DT

Other

430 DT

Then:

Daily average

41 DT

Previous month

1,325 DT

Difference

85 DT less

The user understands spending without analyzing a spreadsheet.

# 66. Mistake Scenario

User records:

120 DT

but meant:

12 DT

Flow:

Activity

↓

120 DT

↓

Edit

↓

12 DT

↓

Save

All related calculations update immediately.

---

## 67. Wrong Category Scenario

User records:

Coffee  
Other

They change:

Other  
→ Food

The application should remember the correction.

Future Coffee expenses should suggest Food.

---

## 68. No Internet Scenario

No internet  
↓  
Add expense  
↓  
Save locally  
↓  
Home updates

Later:

Internet returns  
↓  
Background sync  
↓  
Supabase updated

No user intervention.

---

## 69. New Month Scenario

At budget reset:

New budget period

The Home screen shows:

1,500 DT  
left this month

0 DT  
spent today

Historical data remains accessible through Activity and Insights.

---

## 70. Product Boundaries

Do not add these to V1:

- Investment tracking
- Stocks
- Cryptocurrency
- Loans
- Credit score
- Bank account aggregation
- Complex financial goals
- Debt management
- Tax calculation
- Business accounting
- Multi-currency accounting
- Shared family budgets
- Complex recurring payment management
- Advanced AI financial advice

These features increase complexity and move the product away from the core promise.

---

# 71. V1 Feature Set

Required:

1. Budget setup
2. Expense creation
3. Income creation
4. Expense history
5. Expense editing
6. Expense deletion
7. Categories
8. Automatic category suggestions
9. Monthly budget
10. Daily allowance
11. Home dashboard
12. Insights
13. SwiftData local storage
14. Supabase sync
15. Authentication
16. App Intents
17. Siri
18. Shortcuts
19. Home Screen widget
20. Lock Screen widget
21. Control Center action
22. Dark Mode
23. Accessibility

Optional V1:

24. Voice expense entry
25. Dynamic Island
26. Live Activity
27. Spotlight

Future:

28. NFC
29. Apple Watch
30. Automatic recurring expense detection
31. Advanced AI parsing
32. Smart spending insights
33. Export

## 72. Technical Architecture

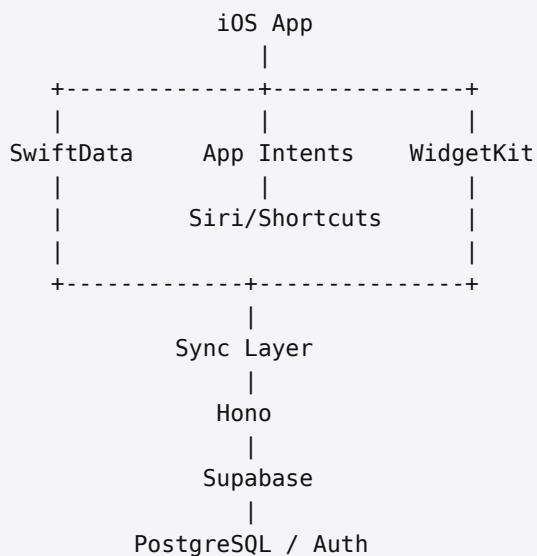
### iOS

Swift  
SwiftUI  
SwiftData  
App Intents  
WidgetKit  
ActivityKit  
UserNotifications  
Core Spotlight

### Backend

Hono  
Supabase  
PostgreSQL  
Supabase Auth  
Supabase Storage

### Architecture



---

## 73. Local-First Sync Model

Every user action should follow:

```
User action
↓
Validate
↓
Save locally
↓
Update UI
↓
Queue sync
↓
Send to backend
↓
Supabase
```

Never:

```
User action
↓
Network request
↓
Wait
↓
Update UI
```

The second architecture would make the app feel slower and less reliable.

---

## 74. API Principles

Keep the API small.

Possible endpoints:

```
POST   /expenses
GET    /expenses
PATCH /expenses/:id
DELETE /expenses/:id

POST   /income
GET    /income

GET    /budget
PATCH /budget

GET    /insights
POST   /ai/parse-expense
```

Avoid creating endpoints for every UI action.

---

## 75. Security

Every user-owned resource must be associated with a user ID.

Supabase Row Level Security must prevent users from accessing another user's:

- Expenses
- Income
- Budget
- Categories
- Preferences

Server-side privileged operations should remain behind Hono.

Never expose privileged Supabase credentials in the iOS application.

---

## 76. Monetization Direction

Do not put monetization in the primary expense flow.

Potential future premium features:

iCloud / advanced sync  
Advanced insights  
AI expense parsing  
Recurring expense detection  
Export  
Multiple budgets  
Apple Watch  
Advanced widgets

The free experience should remain useful.

The product should not lock the fundamental expense-entry experience behind a subscription.

---

## 77. Analytics

Track product behavior without collecting unnecessary financial details.

Useful product events:

```
onboarding_completed  
expense_created  
expense_edited  
expense_deleted  
income_created  
voice_entry_used  
shortcut_used  
widget_opened  
control_used  
insight_viewed
```

Avoid storing raw financial values in analytics systems unless there is a clear product reason.

---

## 78. Success Metrics

Primary:

Time to record an expense

Target:

≤ 3 seconds

Secondary:

Expenses recorded per active user

Weekly active users

7-day retention

30-day retention

Percentage of expenses recorded through system integrations

The last metric is important because the product's differentiation is not simply expense tracking.

The goal is to make expense tracking available everywhere on iOS.

---

## 79. UX Quality Bar

Before shipping any feature, ask:

Does this reduce friction?

Does this make spending easier to understand?

Does this use an iOS capability meaningfully?

Does this remove a manual calculation?

Does this require configuration?

Does this add visual clutter?

Does the user need this?

If a feature adds complexity without improving one of these areas, defer it.

---

## 80. Product Identity

The product should not compete by having the most financial features.

The identity is:

The simplest personal budget app for iPhone.

The application should make the user feel:

I spend money.  
I record it instantly.  
I always know where I stand.

The primary product advantage should be the combination of:

Extreme simplicity  
+  
Native iOS integration  
+  
Local-first speed  
+  
Automatic calculations  
+  
Minimal UI

---

## 81. Final UX Rule

At every point in the product, prioritize this order:

1. Speed
2. Clarity
3. Simplicity
4. Native iOS integration
5. Useful automation
6. Visual polish
7. Advanced functionality

Do not sacrifice the first five to add functionality.

The application should remain simple even as the feature set grows.